

Crash Surfing: Momento III

Marcos Castrillón Velásquez

Luis Andrés Castillo Chicaiza

Junio de 2026

*Universidad de Antioquia
Departamento de Ingeniería Electrónica*

Resumen

Crash Surfing es un videojuego de supervivencia desarrollado en C++ y Qt con dos niveles diferenciados. El proyecto integra interfaz gráfica, administración de estados, colisiones, sprites, sonido y modelos físicos. En el Nivel 1, el jugador controla a Crash sobre una tabla de surf en un entorno lateral donde debe evitar obstáculos, recolectar mangos y sobrevivir a los proyectiles del Dr. Neo Cortex. En el Nivel 2, la dinámica cambia a una vista frontal en la que el reto principal es resistir la atracción de un remolino, recolectar mangos para obtener resistencia y evitar las amenazas del escenario.

El desarrollo se realizó con programación orientada a objetos, usando herencia propia, memoria dinámica y separación entre la lógica del juego y la capa gráfica. El resultado es un videojuego funcional, con niveles seleccionables, pausa, dificultad configurable y una evolución clara desde la idea inicial.

1. Introducción

Crash Surfing es un videojuego desarrollado como proyecto final de Informática 2, utilizando conceptos de programación orientada a objetos, física aplicada, manejo de interfaces gráficas y diseño de sistemas interactivos. La propuesta está inspirada en el universo de Crash Bandicoot y presenta dos niveles con dinámicas distintas: uno lateral y otro frontal, cada uno con su propia lógica de movimiento, obstáculos y condiciones de victoria.

El sistema incluye selección de nivel, pausa, dificultad, enemigos con comportamiento autónomo, efectos de colisión, sonido y una arquitectura dividida entre la capa lógica y la capa gráfica.

2. Planteamiento del problema

En este proyecto, el reto consistió en construir una experiencia jugable que no solo mostrara movimiento y colisiones, sino que también incorporara modelos físicos

diferenciados, agentes inteligentes y una progresión de dificultad perceptible para el usuario.

Además, el proyecto debía cumplir con requisitos concretos: uso de programación orientada a objetos, memoria dinámica, al menos tres modelos físicos, interfaz gráfica en Qt, diagrama de clases y documentación de la evolución del desarrollo en varios momentos. Por ello, el trabajo no se limitó a programar un juego, sino a diseñar un sistema completo.

3. Objetivos

3.1. Objetivo general

Desarrollar Crash surfing implementando modelos físicos, agente inteligente, sistema de dificultad, interfaz gráfica funcional y usar programación orientada a objetos.

3.2. Objetivos específicos

- Implementar dos niveles con dinámicas diferentes, uno lateral y otro frontal, con al menos tres modelos físicos relevantes para la jugabilidad.
- Diseñar y utilizar clases propias con herencia y memoria dinámica.
- Incorporar agente inteligente con comportamiento distintivo.
- Incluir un sistema de dificultad que modifique el reto del jugador.
- Integrar efectos de sonido y recursos gráficos para mejorar la experiencia.

4. Metodología de desarrollo

El desarrollo de *Crash Surfing* se realizó siguiendo la estructura de entregas parciales definida por los momentos del curso. En una primera etapa se estableció el concepto general del juego, el universo temático, la

división por niveles y los modelos físicos inicialmente propuestos. Posteriormente, en el segundo momento, se ajustaron las mecánicas para hacer viable su implementación en Qt, tanto en la complejidad de algunas físicas como en la organización de clases y responsabilidades del sistema.

La implementación se abordó mediante una arquitectura POO, separando el backend de simulación del frontend gráfico. La lógica del juego se concentró en clases encargadas de actualizar entidades, controlar estados, procesar colisiones y aplicar reglas de victoria o derrota, mientras que la capa gráfica construyó la escena, dibujó sprites y manejó las entradas del teclado. Esta separación se observa en la relación entre ‘Game’ y ‘MainWindow’ para el primer nivel, y entre ‘GameLevel2’ y el anterior mencionado para el segundo.

El bucle principal se implementó mediante ‘QTimer’, lo que permitió actualizar el juego en intervalos constantes. Con ello, se integraron movimientos, física, inteligencia artificial, HUD, animaciones simples y validaciones de estado.

5. Arquitectura y diseño del sistema

5.1. Vista general

La arquitectura del proyecto se diseñó a partir de una división clara entre la lógica del juego y la visualización en Qt.

En el Nivel 1, la clase **Game** actúa como núcleo de la simulación. Esta clase administra al jugador, al enemigo, obstáculos y items, además de controlar la actualización temporal, las colisiones, la generación de proyectiles y los cambios de estado del nivel. Por su parte, **MainWindow** se encarga de construir la escena, cargar sprites, crear objetos visuales, procesar eventos de teclado y sincronizar cada elemento gráfico con su correspondiente objeto lógico.

En el Nivel 2, el mismo enfoque se mantiene mediante **GameLevel2** y **MainWindow**. La primera concentra la lógica del nivel, el temporizador de supervivencia, el manejo del jugador y el comportamiento del remolino, mientras que la segunda organiza la escena, la interfaz visual, la representación del vórtice y de la zona de muerte.

5.2. Organización por clases

La estructura del sistema se basa en una jerarquía de entidades. Se definió una clase base **Entity** para modelar de forma homogénea al jugador, los enemigos, los obstáculos y otros elementos interactivos

del juego. Esta decisión permite reutilizar atributos como posición, velocidad, dimensiones y estado de actividad, y facilita el manejo de colisiones y actualizaciones.

Con ello, el jugador y los enemigos extienden el comportamiento general con reglas propias, es decir, el jugador del Nivel 1 se administra a través de **Player**, mientras que el segundo nivel usa **Player2**. De forma similar, el sistema contempla componentes específicos del nivel, como el remolino modelado mediante la clase **Whirlpool**.

La representación gráfica usa **QGraphicsScene**, **QGraphicsView**, **QGraphicsPixmapItem**, **QGraphicsTextItem** y **QGraphicsRectItem**. Esto permite construir una escena compuesta por sprites, textos de interfaz, hitbox de depuración y capas de fondo con distintos niveles de profundidad.

5.3. Estados del juego

El sistema maneja estados explícitos de ejecución para organizar el flujo del juego. En este caso se utilizan estados como ‘MENU’, ‘PAUSED’, ‘PLAYING’, ‘GAMEOVER’ y ‘LEVELCOMPLETE’. Esta organización permite decidir cuándo actualizar la física, cuándo aceptar ciertas entradas del usuario y cuándo mostrar mensajes de interfaz.

Sobre estos estados se implementan acciones para iniciar la partida, reiniciar después de perder, mostrar el HUD únicamente cuando el juego está activo y detener el avance cuando se alcanza una condición.

5.4. Memoria dinámica y modularidad

Para cumplir con los requerimientos del proyecto, se crean dinámicamente el jugador, el enemigo, obstáculos, items, escenas, temporizadores y múltiples objetos visuales, y además se liberan en destructores. La modularidad se evidencia al separar funciones de inicialización, creación de entidades visuales, limpieza de recursos y actualización de interfaz.

5.5. Diagrama resumen

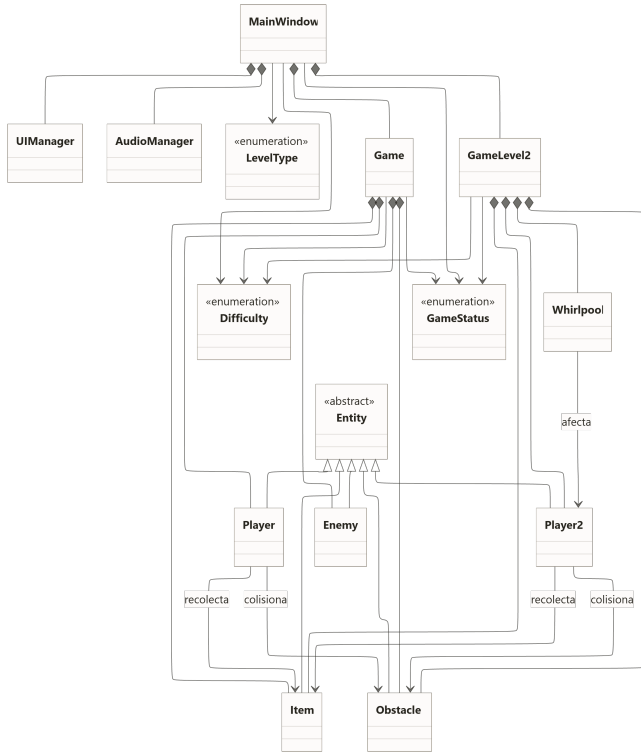


Figura 1: Diagrama resumen de clases.

6. Modelos físicos implementados

6.1. Nivel 1

En el Nivel 1 se implementó el movimiento parabólico del salto del jugador. Este comportamiento es de la clase **Player** y se actualiza en la función `update(float dt)`, donde se aplica una velocidad vertical inicial negativa al ejecutar `jump()` y luego se incorpora la aceleración gravitacional mediante la relación

$$v_y(t) = v_{y0} + gt$$

y el desplazamiento vertical

$$y(t) = y_0 + v_y(t) t$$

También se conserva el movimiento oscilatorio armónico en ciertos obstáculos del escenario, especialmente en elementos móviles como sierras y el Doc Neo Cortex. Este comportamiento está implementado en la clase **Obstacle**, dentro de la función `update(float dt)`, mediante una variación sinusoidal de la posición vertical del objeto, de la forma

$$y(t) = A \sin(\omega t + \varphi) + y_0$$

o una expresión equivalente basada en `std::sin()`, lo que produce un desplazamiento periódico.

La detección de colisiones se realiza con cajas delimitadoras rectangulares, comparando la posición y dimensiones de cada entidad. Este proceso está concentrado en la función `Game::checkCollisions()`, donde se verifica la intersección entre el jugador y los obstáculos o ítems. Cuando Crash colisiona con un obstáculo, la función `Player::onCollision(Entity* other)` aplica daño mediante `takeDamage()` y activa la invulnerabilidad temporal con `startInvincibility()`.

La recolección de mangos también se gestiona desde `Player::onCollision(Entity* other)`, específicamente cuando el objeto de tipo **Item** corresponde a una fruta. En ese caso, la función `collectItem()` incrementa el contador de progreso del jugador. Además, el enemigo del nivel, modelado en la clase **Enemy**, utiliza una lógica de disparo predictivo en la función `Game::spawnProjectile()`, donde se estima la posición futura del jugador y se genera un proyectil con trayectoria calculada.

6.2. Nivel 2

En el Nivel 2 la física principal está asociada al remolino, modelado por la clase **Whirlpool**. Su comportamiento se implementa en la función `computeFrameVelocity(const Player2* player, float dt)`, donde se calcula un vector de atracción hacia el centro remolino. La magnitud de dicha fuerza depende de la distancia entre el jugador y el centro, por lo que el efecto puede expresarse como una fuerza radial variable del tipo

$$\vec{F}_r = F(d) \hat{r}$$

donde $F(d)$ representa la intensidad de atracción según la distancia. En la implementación, esa intensidad se evalúa en `getForceMagnitude(float distance)`, que incrementa el arrastre a medida que el jugador se acerca al centro.

Además de la componente radial, el remolino incorpora una componente tangencial que produce un efecto de giro alrededor del centro. Esto genera una sensación de succión más natural y se aproxima al movimiento de un vórtice. El resultado final se aplica al jugador como una velocidad externa por cuadro, usando la función `Player2::setExternalVelocity(const QVector2D& externalVel)`.

El jugador del Nivel 2, modelado por la clase **Player2**, responde a esta física en `Player2::update(float dt)`. Allí se combinan la dirección de entrada del usuario con

la velocidad externa producida por el remolino. La actualización puede interpretarse como

$$\vec{v} = \vec{v}_{\text{input}} + \vec{v}_{\text{externa}}$$

lo que permite que el movimiento manual compita con la atracción del entorno. Además, la función `getWhirlpoolResistance()` calcula la resistencia adquirida por el jugador a partir de los mangos recolectados, limitando su efecto con un máximo efectivo.

Las colisiones con cajas y otros obstáculos se procesan en `Player2::onCollision(Entity* e)`. Cuando el jugador impacta un objeto sólido, puede recibir daño mediante `takeDamage()` y, en el caso de ciertos obstáculos, activar una ralentización con `applySlow()`. Si el jugador entra en la zona central del vórtice, verificada mediante `Whirlpool::isInsideDeathZone(Player2* player)`, se produce la derrota inmediata.

7. Desarrollo e implementación

La clase ‘Game’ administra memoria dinámica para el jugador, el enemigo, las entidades y los objetos del nivel. Al reiniciar, se liberan los recursos anteriores, se restablece el estado y se recarga el nivel. En el juego, esto permite reinicios limpios sin reconstruir toda la aplicación, y además cumple con el requisito de trabajar con punteros y objetos creados dinámicamente.

La lógica del enemigo del Nivel 1 incluye una estrategia de disparo basada en predicción. El sistema estima la posición futura del jugador usando su velocidad y una ventana de anticipación, ajusta una probabilidad de ataque según la distancia y la posición del personaje, y luego genera un proyectil con componente horizontal y vertical.

En el Nivel 2, el remolino como enemigo, ejerce la atracción según la distancia de Crash hacia su centro, ejerciendo una mayor fuerza entre más se acerque.

7.1. Cronograma

El desarrollo del proyecto tuvo una duración aproximada de 60 horas de trabajo, distribuidas entre análisis, implementación, pruebas y depuración. En una primera etapa se dedicaron entre 3 y 5 horas al diseño inicial del proyecto y al análisis de requisitos. Posteriormente, la implementación base del Nivel 1 tomó entre 8 y 12 horas, seguida por la construcción de la física del salto parabólico, con 2 a 3 horas adicionales. El sistema de colisiones y vidas requirió entre 2 y 4 horas, mientras que la generación de obstáculos y frutas tomó de 2 a 3 horas. El agente autónomo enemigo demandó entre 4 y 6 horas de desarrollo, y el Nivel 2 junto con la implementación del remolino representaron entre 6 y 8 horas de trabajo.

A esto se sumaron entre 1 y 2 horas para el sistema de dificultad, 2 a 3 horas para audio y multimedia, y entre 5 y 8 horas para la interfaz en Qt, incluyendo HUD, menús y estados. Finalmente, la corrección de errores y pruebas consumió entre 6 y 10 horas, la optimización, comentarios y limpieza final entre 3 y 5 horas, y la generación del ejecutable y despliegue entre 1 y 2 horas, completando así el total estimado de 60 horas de desarrollo.

8. Agente inteligente y dificultad

En el Nivel 1, *Neo Cortex* controla el disparo de proyectiles hacia Crash. El sistema observa la posición, la velocidad y la cercanía del jugador para decidir cuándo vale la pena atacar. Además, la función de disparo usa una predicción de trayectoria y un error aleatorio controlado, lo que evita que el comportamiento sea completamente determinista y mejora la sensación de persecución.

En el Nivel 2, la amenaza principal no es un enemigo móvil, sino el remolino y su fuerza de atracción. El reto consiste en sobrevivir el tiempo requerido mientras se evita el centro de succión, se sortean troncos que ralentizan al jugador y se aprovechan los mangos para aumentar la resistencia y cumplir con la cantidad requerida.

La dificultad modifica el reto de forma directa. En el Nivel 1, ajusta la probabilidad y precisión de los disparos enemigos además de cambiar la cantidad de frutas a recolectar; en el Nivel 2, determina la cantidad de mangos necesarios para ganar con un tiempo diferente y más fuerza del remolino, haciendo que el modo difícil exija una gestión más precisa del movimiento y del riesgo.

9. Pruebas y validación

Durante el desarrollo se realizaron pruebas funcionales sobre los dos niveles, con énfasis en movimiento, colisiones, selección de estados, reinicio y control de dificultad. En el Nivel 1 se verificó que Crash pudiera saltar correctamente, recolectar mangos, perder vidas por colisión con obstáculos y activar invulnerabilidad temporal después de recibir daño. También se comprobó que el enemigo mantuviera una lógica de ataque basada en la posición y el comportamiento del jugador.

En el Nivel 2 se validó la respuesta del jugador frente a la atracción del remolino, la pérdida de vida por colisión con troncos y la derrota instantánea al llegar al centro del vórtice.

10. Cambios respecto a los momentos anteriores

El proyecto experimentó cambios desde la primera propuesta hasta la versión final. En el momento inicial se planteó un universo general basado en Crash Bandicoot, con dos niveles, obstáculos, recolección de ítems y modelos físicos como movimiento parabólico, oscilatorio y fuerza centrípeta.

En el segundo momento se realizaron ajustes para asegurar la viabilidad técnica en Qt. Se simplificó el Nivel 1 eliminando el modelado ondulatorio del agua y se redujo la complejidad del Nivel 2 al dejar una fuerza centrípeta continua en lugar de vectores de corriente variables.

En la versión final, el Nivel 1 se construyó con una lógica de desplazamiento lateral, salto parabólico, obstáculos, recolección de mangos, invulnerabilidad y un enemigo con disparo predictivo. El Nivel 2 quedó como una arena frontal con remolino, resistencia obtenida por recolección de ítems, daño por colisiones, ralentización y supervivencia por tiempo.

Otro cambio importante fue la consolidación de clases modulares. En lugar de concentrar toda la lógica en una sola ventana, se separaron funciones de simulación, interfaz y sonido en clases especializadas como Game, GameLevel2, UIManager y AudioManager.

11. Resultados y discusión

El resultado final es un videojuego funcional con dos niveles seleccionables, pausa, dificultad y mecánicas diferenciadas. La implementación logró combinar el universo de Crash Surfing con una arquitectura orientada a objetos y un ejecutable. Esto permitió construir una experiencia jugable que cumple con los requisitos técnicos del proyecto y conserva coherencia entre diseño, física e interacción.

Desde el punto de vista de programación, el proyecto demuestra que es posible integrar modelos físicos simples pero útiles dentro de una aplicación gráfica sin tanta complejidad. En el Nivel 1, la combinación de salto parabólico, colisiones e IA enemiga produce un reto constante. En el Nivel 2, el remolino y la supervivencia por tiempo obligan a una gestión distinta del movimiento y del riesgo.

Para finalizar, se observa que parte de la propuesta inicial fue ajustada para favorecer la estabilidad. Esa reducción de complejidad no debilita el proyecto; al contrario, se prioriza que el sistema funcione de forma consistente por encima de mantener todas las ideas originales sin ajuste.

12. Referencias

- [1] Unity Technologies, “Experiencia de juego en 2D - Manual de Unity,” disponible en: <https://docs.unity3d.com/es/530/Manual/Overview2D.html>.
- [2] Universitat Oberta de Catalunya, “Herencia (relaciones entre clases),” disponible en: <https://openaccess.uoc.edu/server/api/core/bitstreams/127e4b91-7d31-4c96-8cde-4939f52f9a58/content>.
- [3] Refactoring.Guru, “Patrones de diseño / Design patterns,” disponible en: <https://refactoring.guru/es/design-patterns>.
- [4] Qt Forum, “How to play sounds using QMediaPlayer,” disponible en: <https://forum.qt.io/>.

13. Recursos Multimedia

- **Trailer del videojuego:**
https://www.youtube.com/watch?v=n4p_DCUF10I
- **Video de soporte y demostración:**
<https://youtu.be/n0BWvjRgXYg>

14. Anexos



Figura 2: Pantalla de inicio.

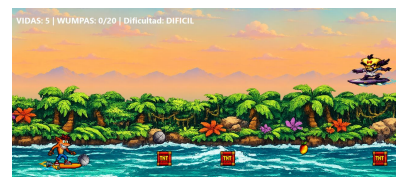


Figura 3: Nivel 1

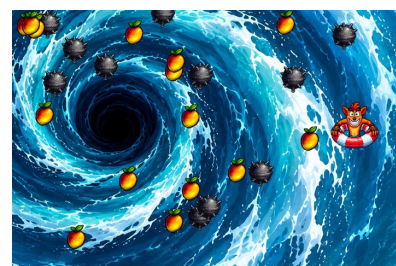


Figura 4: Nivel 2.

